

Endsem

● Graded

Student

MAHAARAJAN J

Total Points

81.5 / 100 pts

Question 1

1a

2 / 2 pts

✓ - 0 pts Correct

Question 2

1b

Resolved 1.5 / 1.5 pts

✓ - 1.5 pts Incorrect answer

💬 + 1.5 pts Point adjustment

🔄 Regrade Request

Submitted on: May 08

Respected ma'am,
The answer to this question is 6.4×10^8 B/s... which i approximated as 640MB/s as 1K is 1024 which is approximately 1000 which should be acceptable... please do consider my request and give me credit for this question

Thanking You

fixed

Reviewed on: May 11

Question 3

1c

1.5 / 1.5 pts

✓ - 0 pts Correct

Question 4

2a

2 / 2 pts

✓ - 0 pts Correct

Question 5

2b

4 / 4 pts

✓ - 0 pts Correct

Question 6

2c

2 / 2 pts

✓ - 0 pts Correct

Question 7

2d

4 / 4 pts

✓ - 0 pts Correct

Question 8

3

7 / 7 pts

✓ - 0 pts Correct

Question 9

4

8 / 8 pts

✓ - 0 pts Correct

Question 10

5a

2 / 2 pts

✓ - 0 pts Correct

Question 11

5b

2 / 2 pts

✓ - 0 pts Correct

Question 12

5c

2 / 2 pts

✓ - 0 pts Correct

Question 13

5d

5 / 5 pts

✓ - 0 pts Correct

Question 14

6

4 / 5 pts

✓ + 4 pts Satisfactory

Question 15

7

1.5 / 8 pts

✓ - 6.5 pts Attempted to answer

Question 16

8a

4 / 4 pts

✓ - 0 pts Correct

Question 17

8b

4 / 4 pts

✓ - 0 pts Correct

Question 18

8c

2 / 2 pts

✓ - 0 pts Correct

Question 19

8d

Resolved 1 / 3 pts

✓ - 3 pts Incorrect

💬 + 1 pt Point adjustment

🔄 Regrade Request

Submitted on: May 08

Respected ma'am,

I have written the expression perfectly correctly and made a mistake only in the computation of the final answer. Please do consider giving partial marks for the same.

Thanking you

given

Reviewed on: May 11

Question 20

8e

2 / 2 pts

✓ - 0 pts Correct

Question 21

8f

3 / 3 pts

✓ - 0 pts Correct

Question 22

9a

3 / 4 pts

✓ - 1 pt Mistake

Question 23

9b

4 / 4 pts

✓ - 0 pts Correct

Question 24

9c

4 / 4 pts

✓ - 0 pts Correct

Question 25

9d

2 / 4 pts

✓ - 2 pts Errors

✓ - 6 pts Incomplete

🔄 Regrade Request

Submitted on: May 08

Respected ma'am,

Could you please provide a rubrik for this question and explain the credit given... I feel I have written enough.... Please do check

Thanking You

Partial marks already given for incomplete and incorrect answer for collective I/O. Same rubric for all.

Reviewed on: May 11

Roll number 720600Name MAHAARASAN J**PLEASE SWITCH OFF YOUR MOBILE PHONE AND KEEP IT IN YOUR BAG**

- Write your name and roll number, your paper will not be evaluated if these are missing.
 - Total time is 180 minutes. Total marks = 100.
 - All questions are compulsory. Marks are mentioned in parenthesis. There is no partial marking in programming questions.
 - Write your name and roll number on rough sheets as well.
 - You **must** answer all questions on the question paper. Ample space has been provided below each question. You may use extra sheets for rough work only. These will not be collected.
 - **Legible** and neat answers will be graded only; no marks will be awarded for illegible text.
 - The MPI function prototypes are given on the last page.
 - Read the questions carefully before answering.
 - All clarifications must be directed to the instructor only.
 - Extra points will not be given for being verbose, so answer to the point, without being abrupt.
 - Please do not leave the exam hall before we finish counting.
- DO NOT OPEN THE QUESTION PAPER BEFORE THE SCHEDULED TIME. IF YOU ARE FOUND DOING SO, 5 MARKS WILL BE DEDUCTED AND YOU WILL LOSE 5 MINUTES.

Marks (To be filled in by the instructor)

1a	1b	1c	2a	2b	2c	2d	3	4	5a	5b
2	1.5	1.5	2	4	2	4	7	8	2	2
5c	5d	6	7	8a	8b	8c	8d	8e	8f	9a
2	5	5	8	4	4	2	3	2	3	4
9b	9c	9d	10	Total						
4	4	4	10	100						

I pledge to answer all questions by myself without taking any external help.

Selvarajan (Sign your name)

Q1. Understand the below code and answer the below questions. (2+1.5+1.5 marks)

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include "mpi.h"
4
5 int main( int argc, char *argv[])
6 {
7     int N=atoi(argv[1]), myrank, size, count[2], number;
8     MPI_Status status[2]; MPI_Request request[2];
9     int arr[2][N], recvarr[N];
10
11     MPI_Init(&argc, &argv);
12     MPI_Comm_rank (MPI_COMM_WORLD, &myrank);
13     MPI_Comm_size (MPI_COMM_WORLD, &size);
14
15     if (myrank == 0) {
16         MPI_Isend(arr[0], N, MPI_INT, 1, 98, MPI_COMM_WORLD, &request[0]);
17         MPI_Isend(arr[1], N, MPI_INT, 1, 99, MPI_COMM_WORLD, &request[1]);
18     }
19     else if (myrank == 1) {
20         MPI_Recv(recvarr, 2*N, MPI_INT, 0, 98, MPI_COMM_WORLD, &status[0]);
21         MPI_Recv(recvarr, 2*N, MPI_INT, 0, 99, MPI_COMM_WORLD, &status[1]);
22         MPI_Get_count (&status[0], MPI_INT, &count[0]);
23         MPI_Get_count (&status[1], MPI_INT, &count[1]);
24     }
25
26     if (myrank == 0) MPI_Waitall (number, request, status);
27
28     MPI_Finalize();
29     return 0;
30 }

```

- (a) What is the bug in the code? Will it give runtime or compile-time error? Mention the fix. number must be initialised to 2. Runtime error as number will have some garbage value in the invocation of Waitall. Fix: number=2; after declaration

Assuming, you have fixed the code and it runs without any errors, answer the below questions:

- (b) What is the achieved communication bandwidth if the total communication time is 1.6 s for the execution `mpirun -np 2 ./a.out 128000000` (size of integer = 4 bytes) 640 MB/s
- (c) What is the total communication time for the execution (`mpirun -np 2 ./a.out 104857600`) if the network bandwidth between 0 to 1 is 32 Mbps 250 200 s

Q2. Marks: 2+4+2+4

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include "mpi.h"
4
5 int main (int argc, char *argv[])
6 {
7     int myrank, size;
8     MPI_Status status1, status2;
9     MPI_Request request1, request2;
10    double sTime, eTime, time;
11
12    MPI_Init(&argc, &argv);
13
14    int count = atoi (argv[1]);
15    int buf[count];
16
17    MPI_Comm_rank(MPI_COMM_WORLD, &myrank);
18    MPI_Comm_size(MPI_COMM_WORLD, &size);
19    if (size%2 != 0) return 0;
20
21    for (int i=0; i<count; i++) buf[i] = myrank+i;
22
23    sTime = MPI_Wtime();
24    if (myrank%2 == 0) {
25        MPI_Send (buf, count, MPI_INT, myrank+1, myrank+1, MPI_COMM_WORLD);
26        MPI_Send (buf, count, MPI_INT, myrank-1, myrank-1, MPI_COMM_WORLD);
27    }
28    else if (myrank%2 != 0) {
29        MPI_Irecv (buf, count, MPI_INT, myrank-1, myrank, MPI_COMM_WORLD, &request1);
30        MPI_Irecv (buf, count, MPI_INT, myrank+1, myrank, MPI_COMM_WORLD, &request2);
31        MPI_Wait (&request1, &status1);
32        MPI_Wait (&request2, &status2);
33    }
34    eTime = MPI_Wtime();
35
36    MPI_Finalize();
37    return 0;
38
39 }
40
```

The intended communication pattern in the above program: even ranks send data to their right and left neighbouring ranks if they exist, similarly odd ranks receive from left and right neighbours if they exist.

a. What error(s) will you encounter for the following execution: `mpirun -np 4 ./a.out 400`

- In the execution for rank 0 there is an MPI-send to $\text{myrank}-1 = -1$ which is an invalid rank, gives a runtime error
- for rank=3 there is an I-recv from rank=4 which does not exist ($\&\text{myrank}+1$) again gives runtime error.

b. Mention the bugs with line numbers and their exact fixes with line numbers (has to be a syntactically correct C code snippet). Your suggested fixes should require the most minimal changes in the given code. # size must be even

the bug is that it is not checked if neighbouring ranks exist or not

~~20~~ 26: if (myrank != 0) MPI_Send(buf, count, MPI_INT, myrank-1, myrank-1, MPI_COMM_WORLD);
 30: if (myrank != size-1) MPI_Recv(buf, count, MPI_INT, myrank+1, myrank, MPI_COMM_WORLD, &request2);
 32: if (myrank != size-1) MPI_Wait(&request2, &status2);

* size is always even as we return if size is odd $\therefore$ last rank is odd hence it receives from rank-1 only.

c. Which ranks may take the longest communication time for the below execution and why:
 mpirun -np 20 -hosts host1:10,host2:10 ./a.out 40000000 (assume that each host has 10 cores)

are expected to
 Ranks 9 and 10 will take the longest communication time as they are neighbours on different nodes. Hence they perform internode communication whereas all other communication pairs are intranode. Hence as internode send, rec expected to take longer $\therefore$ ranks 9, 10 expected to take longest time.

d. Sort the below executions in increasing order of execution time:

- (i) mpirun -np 40 -hosts csews10:10,csews20:10,csews21:10,csews24:10 ./a.out 104857600
- (ii) mpirun -np 40 -hosts csews10:10,csews20:10,csews21:10,csews24:10 ./a.out 1048576
- (iii) mpirun -np 40 -hosts csews10,csews20,csews21,csews24 ./a.out 104857600
- (iv) mpirun -np 10 ./a.out 1048576

Mention (i) (ii) (iii) (iv) below in the sorted order (increasing). Explain your answer.

(iv) < (ii) < (i) < (iii). (iv) is the least as smaller data size and all comm are intranode. Then comes (ii) as smaller data size communicated but 3 internode links/communications. Then we have (i) as larger data size, 3 internode comm, finally (iii) as RR process placement all comm are intranode along with it being larger data size. Larger data size, more internode comm take more time.

Q3. Understand and complete the code for hierarchical reduce, i.e. fill in the blanks. There is no partial marking for syntax errors. Lines 29 and 46 of your completed code should print the same values (2 decimal places) for numprocs>1. (7 marks)

```

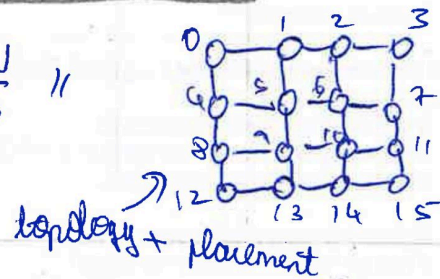
1 #include <sched.h>
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <time.h>
5 #include "mpi.h"
6
7 int main(int argc, char *argv[])
8 {
9     int numprocs, rank, level1_rank, level2_rank, arraySize, local, i;
10    arraySize = atoi (argv[1]);
11    double array[arraySize], sumLocal=0.0, sumGlobal=0.0, sumGlobal1=0.0, time_=0.0, max_time=0.0;
12    MPI_Comm level1_comm, level2_comm;
13
14    MPI_Init(&argc,&argv);
15    MPI_Comm_size(MPI_COMM_WORLD, &numprocs);
16    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
17
18    srand(time(NULL));
19    for (i=0; i<arraySize; i++)
20        array[i] = (double) rand()/((rank+1)*25.25);
21
22    for (i=0; i<arraySize; i++)
23        sumLocal += array[i];
24
25    if (numprocs == 1) sumGlobal = sumLocal;
26    else {
27        // This is the non-hierarchical reduce
28        MPI_Reduce (&sumLocal, &sumGlobal, 1, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);
29        if (!rank) printf ("%lf\n", sumGlobal);
30
31        // Write the hierarchical version of reduce
32        // Step 1: Form the sub-communicators
33        // 0 to P/2-1 belong to one group, P/2 to P-1 belong to another group (level1 groups)
34        int color = rank / (numprocs / 2); //assume numprocs is even
35        MPI_Comm_split (MPI_COMM_WORLD, color, rank, &level1_comm);
36        MPI_Comm_rank (level1_comm, &level1_rank);
37
38        // Rank i of above groups belong to one group (level2 groups)
39        color = rank % (numprocs / 2); ( modulo )
40        MPI_Comm_split (MPI_COMM_WORLD, color, rank, &level2_comm);
41        MPI_Comm_rank (level2_comm, &level2_rank);
42
43        // Step 2: Two-step reduce (final result should be in 0 of MPI_COMM_WORLD)
44        MPI_Reduce (&sumLocal, &sumGlobal1, 1, MPI_DOUBLE, MPI_SUM, 0, level1_comm);
45        MPI_Reduce (&sumGlobal1, &sumGlobal, 1, MPI_DOUBLE, MPI_SUM, 0, level2_comm);
46        if (!rank) printf ("%lf\n", sumGlobal);
47    }
48
49    MPI_Finalize();
50 }

```

Q4. Consider a 2D mesh topology of dimension 4×4 , thus there are 4^2 compute nodes. Assume that 4^2 ranks are placed on this mesh in XY order. Further, assume the latency and bandwidth of each network link in this network are L and B respectively. Derive the communication time of MPI_Bcast (binomial) of N bytes from rank 0 among the 4^2 ranks considering the network topology for every communication in every step. Clearly show all communication steps, hops, and cost calculation for each step, followed by overall cost (time). Assume YX routing in the mesh. (8 marks)

step #	comm ranks	hops	cost	# comm ranks (a,b) ↳ a sends N bytes to b
1	(0,8)	2	$2L + N/B$	
2	(0,4), (8,12)	1	$L + N/B$	
3	(0,2) (4,6) (8,10) (2,14)	2	$3L + N/B$	
4	(0,11) (2,3) (4,5) (6,7) (8,9) (10,11) (2,13) (4,15)	1	$L + N/B$	

* Hence overall cost = $6L + \frac{4N}{B}$ //



* B-cast each step data comm is N

* assume cut through routing $\therefore L' = t_s + L \cdot t_r$
 $\text{B-cast } N/B \text{ only once.}$

Now ignoring t_s for now

2 hops $\Rightarrow L' = 2L$

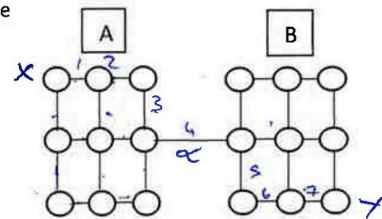
defined L for a link $\Rightarrow t_s + t_r$ (as default 1 hop)

$\therefore$ in step 1, 3 2 hops are taken $\therefore L' = 2L$
 bandwidth the same as cut through

$\rightarrow$ if store forward then cost of 1,3 would become $2L + \frac{2N}{B}$ and total = $6(L + \frac{N}{B})$

Q5. For the network (sub-networks A and B are connected by a link) in the right-side figure, answer the below questions (2+2+2+5)

- What is the bisection width
- What is the diameter
- What is the cost
- Derive the average number of hops in case of naïve Alltoall communication between processes running on nodes in sub-network A



a) 1, as if we disconnect the A-B connecting link (α) then we split the network into 2 equal halves. $bw=1$

b) diameter is the longest path distance which is for node at top left of A to bottom right of B. $diam: 7$

c) 12 in sub-net A, 12 in sub-net B, 1 link $Cost: 25$

d) as all-all in network A we will have all possible communication pairs [assume $\times \times$ process placement of ranks 0 to 8]

1 hop pairs: (0,1), (0,3), (1,2), (1,4), (2,5), (3,4), (3,6), (4,5), (4,7), (5,8), (6,7), (7,8) : 12 pairs

2 hop pairs: (0,2), (0,4), (0,6), (1,3), (1,5), (1,7), (2,4), (2,8), (3,5), (3,7), (4,6), (4,8), (5,7), (6,8) : 14 pairs

3 hop pairs: (0,5), (0,7), (1,6), (1,8), (2,3), (2,7), (3,8), (5,6) : 8 pairs

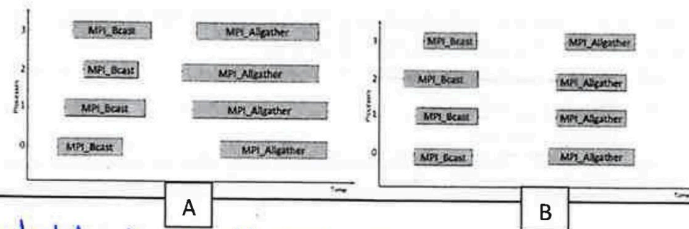
4 hop pairs: (0,8), (2,6) : 2 pairs

Total pairs = 36 [${}^9C_2 = 36$]

Now avg hops = $\frac{12 \times 1 + 14 \times 2 + 8 \times 3 + 2 \times 4}{36}$

$= \frac{72}{36} = 2$ $\therefore$ Avg no of hops = 2

Q6. The figures show two communication profiles of the same application when executed on 4 processes. Mention all your observations from the profiles, compare and contrast, explain the differences and the possible reasons. (5 marks)



• We use same executable in both cases

Observations:

1. data size used in A is longer than that of B as it takes more time to B-cast, Allgather as compared to B from the profiles
2. All gather takes more time than B-cast in both which highlights that some more bandwidth term i.e Allgather has $(\frac{P-1}{B})N$ (assuming N bytes broadcasted, N bytes are gathered from each process)
3. In ~~profile~~ trial A B-cast of 1 took longest whereas in B B-cast of 2 took longest. This could be due to performance variability in completion of sends, receives at 2,1 respectively.
4. All gather is called after B-cast in both cases.
5. data size more in B but not that too much change in B-cast as comp to all gather as the data

term of all-gather is larger with $(\frac{P-1}{B})N$ as opposed to $(\log P) \frac{N}{B}$ in round ring algo -

Q7. Explain and illustrate pipelined segmented MPI_Reduce using recursive doubling for P nodes. Segmented implies multiple MPI_Reduce of shorter data sizes. Total message size = N bytes per process. Segment size = x bytes. Various phases of multiple MPI_Reduce calls can progress in pipelined manner. E.g. step 1 of MPI_Reduce for 3rd segment may overlap with a communication step of 2nd or 1st MPI_Reduce wherever possible, reducing idling. Clearly depict the step number, segment number, and data size for every step and pipeline stages for P=8, N=1 MB, x=128 KB. Clearly number the various stages of the pipeline and the communications steps. (8 marks)

- Now no of segments = N/x

$$= \frac{1024 \cdot 2^{30}}{2^{10} \times 2^7} = 8 \text{ segments}$$
- no of steps required in original all red = $\log_2 8$
 $= 3 \text{ steps}$
- Let us define pipeline stages in the following way.
 Stage 1: includes comm of $0 \leftrightarrow 1, 2 \leftrightarrow 3, 4 \leftrightarrow 5, 6 \leftrightarrow 7$
 Stage 2: includes comm of $0 \leftrightarrow 2, 4 \leftrightarrow 6$
 Stage 3: includes comm of $0 \leftrightarrow 4$
 each segment goes thru these 3 pipeline stages
 Now these 3 stages can be executed together
 Here is a diagram showing the pipeline

	t =	1	2	3	4
seg 1		S1	S2	S3	
seg 2			S1	S2	S3
seg 3				S1	S2 S3
...					

★ Hence for 8 segments to finish parallelly this way

we will need $8 + 2 = 10$ time steps

at $t=1$ seg 1 will be executing stage 1 of pipeline

$t=2$ seg 1 in S_2 , seg 2 in $S_1 \dots$

Now since $0 \leq 2, 0 \leq 1$ comm at same time let us define a diff host for each segment and rotate it like 0, 1, 2, 3, 4, 5, 6, 7 for seg 1
 1, 2, 3, 4, 5, 6, 7, 0 for seg 2
 $\hookrightarrow$ comm in S_1
 $\hookrightarrow$ ~~2, 3, 4, 5, 6, 7~~

Q8. The details of jobs submitted to a supercomputer are shown below. The submit time is given in the second column. The supercomputer has 100 nodes and uses FCFS with backfilling scheduling policy. Deduce the following for the jobs: (a) Start time (column 5) (b) Wait time (column 6) of each job. (c) Calculate the average wait time. (d) Calculate the total idle node-hours from start (time when the first job enters) to end (time when the last job exits). (e) Which are the backfill jobs and why/how? (f) Draw a neat node allocation diagram, indicate job IDs, times, and nodes. (4+4+2+3+2+3)

Job ID	Submit time	Requested # nodes	Requested wall-clock time (hours)	Start time	Wait time (hours)
1	00:00	32	2	00:00	0
2	00:00	64	4	00:00	0
3	00:30	32	3	02:00	01:30 (1.5h)
4	00:30	32	4	04:00	03:30 (3.5h)
BF $\leftarrow$ 5	01:00	4	2	01:00	0
6	01:00	96	1	08:00	07:00 (7h)
BF $\leftarrow$ 7	01:00	64	2	05:00	04:00 (4h)
BF $\leftarrow$ 8	01:30	4	2	03:00	01:30 (1.5h)

(c) Avg wait time = $\frac{1.5 + 1.5 + 3.5 + 4 + 7}{8}$
 $= \frac{17.5}{8}$
 $\approx 2.2 \text{ h.}$

(d) Time supercomp runs = 9h (last job end)

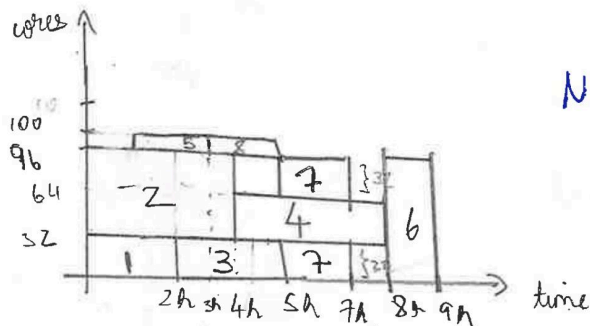
$\therefore$ Total node-hours = 900

$$\begin{aligned} \text{node-hours used} &= 32 \times 2 + 64 \times 4 + 32 \times 3 + 32 \times 4 + 4 \times 2 + 96 \times 1 + \\ &\quad 64 \times 2 + 4 \times 2 \\ &= 768 \end{aligned}$$

$\therefore$ idle hours = 132 node-hours

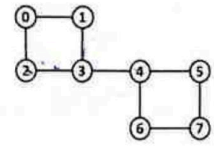
(e) The backfill jobs are 5, 7, 8. i) 5 is backfill as when it enters at 01:00 4 nodes avail, job 3 start time not affected due to 5's sched (starts at 02:00 hrs) (ii) ~~8~~ 8 is backfill as when 5 completes at $t=3h$ 4 nodes avail, start time of job 4 not hindered (iii) lastly 7 is backfill at $t=5h$ as 68 nodes avail, job 6 can only be sched at $t=8h$, 7 completes by $t=7h$ no affect...

(f)



Node allocation diagram

Q9. Analyze MPI_Allgather algorithms on the given network. Rank placements are as shown. Illustrate the step number, data size, hop count for every communication step of (a) ring algorithm and (b) recursive doubling algorithm. (c) Derive and depict the communication matrix for ring algorithm (d) Derive and depict the communication matrix for recursive doubling algorithm. Data size = N bytes per process. (4+4+4+4)



$P=8$

(a)

step	hop count	comm pairs	data size
1	2 (for 1,2 pair max)	all adj pairs $0 \rightarrow 1, 1 \rightarrow 2, \dots, 7 \rightarrow 0$	N
2	2	* we always send N bytes read in prev step in this step to the next neighbour in the ring fashion	N
3	2		N
4	2		N
5	2		N
6	2		N
7	2		N

(b)

step	hop count	data size
1	1	N
2	1	$2 \times N$
3	3	$4 \times N$

* Now step 1 involves $0 \leftrightarrow 1, 2 \leftrightarrow 3, 4 \leftrightarrow 5, 6 \leftrightarrow 7$
N.ee comm

step 2 $\Rightarrow 0 \leftrightarrow 2, 1 \leftrightarrow 3, 4 \leftrightarrow 6, 5 \leftrightarrow 7$ $2 \times N$ comm (0 comm its and 1's data)

step 3 $\Rightarrow 0 \leftrightarrow 4, 1 \leftrightarrow 5, 2 \leftrightarrow 6, 3 \leftrightarrow 7$ all data aggs
comm $4 \times N$

(c)

	0	1	2	3	4	5	6	7
0		α						α
1	α		α					
2		α		α				
3			α		α			
4				α		α		
5					α		α	
6						α		α
7	α						α	

0 comm to 1 in all steps, sends N bytes

$\therefore 7 * N$ bytes comm

let $\alpha = 7 * N$

unfilled entries = 0

* Only neighbors $\begin{pmatrix} 0,1 \\ 1,2 \\ \vdots \\ 7,0 \end{pmatrix}$

communicate α amount of data in whole of All-gather, Rest no communication

(d)

	0	1	2	3	4	5	6	7
0		N	$2N$		$4N$			
1	N			$2N$		$4N$		
2	$2N$			N			$4N$	
3		$2N$	N					$4N$
4	$4N$					N	$2N$	
5		$4N$			N			$2N$
6			$4N$		$2N$			N
7				$4N$		$2N$	N	

deriv in part b

2 comm pair is involved in 1 step alone $\therefore$ just fill data communicated

Q10. Assume a 2D domain ($N \times N$) decomposed to $P \times P$ processes (P rows and P columns). The data is written to a file (XY order) using collective I/O. Explain, compare and contrast two-phase I/O strategy using (a) 1 aggregator/row (b) 1 aggregator/column when the processes are placed sequentially on P nodes (each node has P cores). Also, illustrate the file domains of each aggregator for (a) and (b), you may represent a file using a rectangle such as below (10 marks)

a) In case of 1 aggregator per row that aggregator will have to receive the subdomain of all processes in its row

∴ since Processes are also in x-y order



the extents of domains are contiguous. Hence the aggregators will need to read $\frac{N^2}{P}$ bytes i.e.

i^{th} row agg starts from $(i-1) \frac{N^2}{P}$ to $i \frac{N^2}{P}$ ($i: 0 \rightarrow P$)

the rectangle for phase 1 is

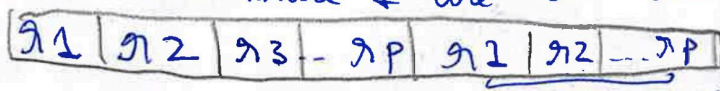


where r_i is the i^{th} row aggregator. ~~Another app is file~~

Phase 2: In phase 2 it scatters amongst members of the row can be done by using

each block is $\frac{N^2}{P}$ sub-com

b) Column agg — non contiguous data accesses hence file domain in phase 1 like as columns non contig



access block is $\left(\frac{N^2}{P^2}\right)$ of length, each agg will access P many

#. Now in (a) more efficient ^{in Phase 1} as number of accesses by the aggregator is less (1 access enough but in b P many accesses) in Phase 1. Phase 2 of both is ~~same~~ ^{similar} as requires scatter among same num of processes (1 across columns scatter, 2 across rows)

* also in Phase 2 as sequential process placement better

to scatter ~~across rows~~ as will result in only intra node communication as (a)

0 1 2 3
4 5 6 7
8 9 10 11
12 13 14 15

⇒ 0 scatters to 0, 1, 2, 3 all in same node but in (b) 0 scatters to 0, 4, 8, 12 which are in different nodes

MPI FUNCTIONS (PROTOTYPES)

Size of integer = 4 bytes, 1 KB = 1024 bytes, 1 MB = 1024 KB

- 1) int **MPI_Comm_rank** (MPI_Comm comm, int *rank)
- 2) int **MPI_Comm_size** (MPI_Comm comm, int *size)
- 3) int **MPI_Get_processor_name**(char *name, int *resultlen)
- 4) int **MPI_Comm_split**(MPI_Comm comm, int color, int key, MPI_Comm * newcomm)
- 5) int **MPI_Group_incl**(MPI_Group group, int n, const int ranks[], MPI_Group *newgroup)
- 6) int **MPI_Comm_create_group**(MPI_Comm comm, MPI_Group group, int tag, MPI_Comm * newcomm)
- 7) int **MPI_Comm_group**(MPI_Comm comm, MPI_Group * group)
- 8) int **MPI_Send** (const void *buf, int count, MPI_Datatype datatype, int dest, int tag, MPI_Comm comm)
- 9) int **MPI_Recv** (void *buf, int count, MPI_Datatype datatype, int source, int tag, MPI_Comm comm, MPI_Status *status)
- 10) int **MPI_Get_count** (const MPI_Status *status, MPI_Datatype datatype, int *count)
- 11) int **MPI_Isend**(const void *buf, int count, MPI_Datatype datatype, int dest, int tag, MPI_Comm comm, MPI_Request *request)
- 12) int **MPI_Irecv**(void *buf, int count, MPI_Datatype datatype, int source, int tag, MPI_Comm comm, MPI_Request *request)
- 13) int **MPI_Wait**(MPI_Request *request, MPI_Status *status)
- 14) int **MPI_Waitall**(int count, MPI_Request array_of_requests[], MPI_Status array_of_statuses[])
- 15) double **MPI_Wtime**(void)
- 16) int **MPI_Bcast**(void *buffer, int count, MPI_Datatype datatype, int root, MPI_Comm comm)
- 17) int **MPI_Gather**(const void *sendbuf, int sendcount, MPI_Datatype sendtype, void *recvbuf, int recvcnt, MPI_Datatype recvtpe, int root, MPI_Comm comm)
- 18) int **MPI_Scatter**(const void *sendbuf, int sendcount, MPI_Datatype sendtype, void *recvbuf, int recvcnt, MPI_Datatype recvtpe, int root, MPI_Comm comm)
- 19) int **MPI_Reduce**(const void *sendbuf, void *recvbuf, int count, MPI_Datatype datatype, MPI_Op op, int root, MPI_Comm comm)
- 20) int **MPI_Allreduce**(const void *sendbuf, void *recvbuf, int count, MPI_Datatype datatype, MPI_Op op, MPI_Comm comm)
- 21) int **MPI_Allgather**(const void *sendbuf, int sendcount, MPI_Datatype sendtype, void *recvbuf, int recvcnt, MPI_Datatype recvtpe, MPI_Comm comm)
- 22) int **MPI_Alltoall**(const void *sendbuf, int sendcount, MPI_Datatype sendtype, void *recvbuf, int recvcnt, MPI_Datatype recvtpe, MPI_Comm comm)
- 23) int **MPI_Pack**(const void *inbuf, int incnt, MPI_Datatype datatype, void *outbuf, int outsize, int *position, MPI_Comm comm)
- 24) int **MPI_Unpack**(const void *inbuf, int insize, int *position, void *outbuf, int outcount, MPI_Datatype datatype, MPI_Comm comm)
- 25) int **MPI_Type_vector**(int count, int blocklength, int stride, MPI_Datatype oldtype, MPI_Datatype * newtype)
- 26) int **MPI_Type_indexed**(int count, const int *array_of_blocklengths, const int *array_of_displacements, MPI_Datatype oldtype, MPI_Datatype *newtype)
- 27) int **MPI_File_read_at**(MPI_File fh, MPI_Offset offset, void *buf, int count, MPI_Datatype datatype, MPI_Status * status)
- 28) int **MPI_File_open**(MPI_Comm comm, ROMIO_CONST char *filename, int amode, MPI_Info info, MPI_File * fh)
- 29) int **MPI_Gatherv**(const void *sendbuf, int sendcount, MPI_Datatype sendtype, void *recvbuf, const int *recvcnts, const int *displs, MPI_Datatype recvtpe, int root, MPI_Comm comm)
- 30) int **MPI_Scatterv**(const void *sendbuf, const int *sendcounts, const int *displs, MPI_Datatype sendtype, void *recvbuf, int recvcnt, MPI_Datatype recvtpe, int root, MPI_Comm comm)
- 31) int **MPI_Allgatherv**(const void *sendbuf, int sendcount, MPI_Datatype sendtype, void *recvbuf, const int *recvcnts, const int *displs, MPI_Datatype recvtpe, MPI_Comm comm)
- 32) int **MPI_Alltoallv**(const void *sendbuf, const int *sendcounts, const int *sdispls, MPI_Datatype sendtype, void *recvbuf, const int *recvcnts, const int *rdispls, MPI_Datatype recvtpe, MPI_Comm comm)
- 33) int **MPI_Ibcast**(void *buffer, int count, MPI_Datatype datatype, int root, MPI_Comm comm, MPI_Request *request)
- 34) int **MPI_Igather**(const void *sendbuf, int sendcount, MPI_Datatype sendtype, void *recvbuf, int recvcnt, MPI_Datatype recvtpe, int root, MPI_Comm comm, MPI_Request *request)
- 35) int **MPI_Iscatter**(const void *sendbuf, int sendcount, MPI_Datatype sendtype, void *recvbuf, int recvcnt, MPI_Datatype recvtpe, int root, MPI_Comm comm, MPI_Request *request)

- 36) int **MPI_allgather**(const void *sendbuf, int sendcount, MPI_Datatype sendtype, void *recvbuf, int recvcount, MPI_Datatype recvtype, MPI_Comm comm, MPI_Request *request)
- 37) int **MPI_ireduce**(const void *sendbuf, void *recvbuf, int count, MPI_Datatype datatype, MPI_Op op, int root, MPI_Comm comm, MPI_Request *request)
- 38) int **MPI_lallreduce**(const void *sendbuf, void *recvbuf, int count, MPI_Datatype datatype, MPI_Op op, MPI_Comm comm, MPI_Request *request)
- 39) int **MPI_lalltoall**(const void *sendbuf, int sendcount, MPI_Datatype sendtype, void *recvbuf, int recvcount, MPI_Datatype recvtype, MPI_Comm comm, MPI_Request *request)
- 40) int **MPI_lgatherv**(const void *sendbuf, int sendcount, MPI_Datatype sendtype, void *recvbuf, const int recvcounts[], const int displs[], MPI_Datatype recvtype, int root, MPI_Comm comm, MPI_Request *request)
- 41) int **MPI_lscatterv**(const void *sendbuf, const int sendcounts[], const int displs[], MPI_Datatype sendtype, void *recvbuf, int recvcount, MPI_Datatype recvtype, int root, MPI_Comm comm, MPI_Request *request)
- 42) int **MPI_lallgatherv**(const void *sendbuf, int sendcount, MPI_Datatype sendtype, void *recvbuf, const int recvcounts[], const int displs[], MPI_Datatype recvtype, MPI_Comm comm, MPI_Request *request)
- 43) int **MPI_lalltoallv**(const void *sendbuf, const int sendcounts[], const int displs[], MPI_Datatype sendtype, void *recvbuf, const int recvcounts[], const int rdispls[], MPI_Datatype recvtype, MPI_Comm comm, MPI_Request *request)
- 44) int **MPI_Type_vector**(int count, int blocklength, int stride, MPI_Datatype oldtype, MPI_Datatype * newtype)
- 45) int **MPI_Type_indexed**(int count, const int *array_of_blocklengths, const int *array_of_displacements, MPI_Datatype oldtype, MPI_Datatype *newtype)
- 46) int **MPI_File_read_at**(MPI_File fh, MPI_Offset offset, void *buf, int count, MPI_Datatype datatype, MPI_Status * status)
- 47) int **MPI_File_read_all**(MPI_File fh, void *buf, int count, MPI_Datatype datatype, MPI_Status *status) Input Parameters
- 48) int **MPI_File_read_at_all**(MPI_File fh, MPI_Offset offset, void *buf, int count, MPI_Datatype datatype, MPI_Status * status)
- 49) int **MPI_File_open**(MPI_Comm comm, ROMIO_CONST char *filename, int amode, MPI_Info info, MPI_File * fh)
- 50) int **MPI_File_close**(MPI_File * fh)

